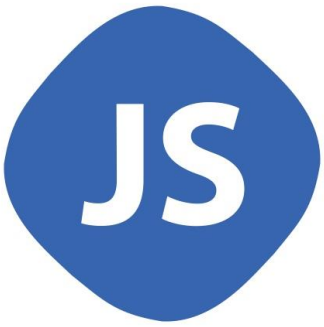


Введение в сценарии JS

- Введение
- Подключение сценариев
- Синхронные, асинхронные и отложенные сценарии
- Синтаксис JavaScript
- EcmaScript

Введение

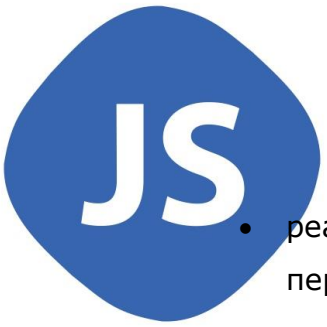
JavaScript — это кросс-платформенный, объектно-ориентированный скриптовый язык, который позволяет веб-разработчику управлять тем, как **ведет** себя веб-страница. Это делает JavaScript принципиально отличным от HTML, который отвечает за **структуру** веб-документа, и CSS, который формирует внешний **вид** веб-страниц.

Программы, написанные на языке JavaScript, называются **скриптами** или **сценариями**. Скрипты JavaScript могут быть вставлены в любое место HTML-документа, и как только браузер отображая код страницы доходит до скрипта – выполняются. Процесс выполнения скрипта называют **интерпретацией**.

> ***Важно***. Если PHP скрипт обрабатывается **на стороне сервера** с помощью PHP интерпретатора, то JavaScript выполняется **в браузере пользователя JavaScript интерпретатором**.

Сегодня каждый браузер поддерживает JavaScript, тем самым делая его языком веба-разработки. Вот некоторые из задач, которые может решать программист с помощью JavaScript:

- проверять правильность заполнения пользовательских HTML-форм;
- менять стили HTML-элементов, скрывать, показывать элементы и т.п.;
- отображать всплывающие и диалоговые окна;



- реагировать на действия посетителя, обрабатывать клики мыши, перемещения курсора и т.п.;
- посылать запросы на сервер и загружать данные без перезагрузки страницы.

Подключение сценариев

Чтобы программа (сценарий) JavaScript запустилась, ее нужно внедрить в HTML-документ. Сценарии можно внедрить в HTML-документ различными стандартными способами:

- поместить код между открывающим и закрывающим тегами **script**;
- поместить скрипты во **внешний файл** (с расширением .js), а затем подключить его к документу HTML;
- поместить код непосредственно в **атрибут события** HTML-элемента.

JavaScript в элементе script

Самый простой способ внедрения JavaScript в HTML-документ – использование тега script. Теги script могут размещаться в **любом** месте документа.

Таким образом, на одной веб-странице могут располагаться сразу несколько сценариев. Выполнение сценариев браузерами происходит по мере их загрузки. Браузер читает HTML-документ сверху вниз и, когда он встречает тег script, рассматривает текст программы как сценарий и **выполняет** его.

> ***Важно***. Оставшийся HTML-контент страницы не загружается и не отображается, пока не будет выполнен **весь код в элементе script**.

В коде листинга example_1 представлен первый сценарий курса на языке программирования JavaScript. Добро пожаловать!

example_1. Выполнение кода скрипта

```
<!-- ...  -->
      <header>
            <h1 align = "center">Введение в JavaScript</h1>
      </header>
      <!-- ... -->
      <script>
            alert("Добро пожаловать в JavaScript!");
      </script>
      <!-- ... -->
      <footer>
            <h3 align = "center">Добро пожаловать на pechora_PROger</h3>
      </footer>
<!-- ... -->
```

Однако считается плохим стилем программирования "разбрасывать" элементы script по всему документу. Гораздо правильнее приучить себя создавать сценарии либо в начале, либо в конце документа. Таким образом, так же, как отделяют CSS от HTML, стараются отделить **сценарии** JavaScript от собственно **верстки** HTML-документа.

example_2. Вынесение кода в тег HEAD

```
<!-- ...  -->
<head>
      <meta http-equiv="Content-Type" content="text/html; charset=utf-
      8">
      <title>Введение в JavaScript</title>
      <script>
            alert("Добро пожаловать в JavaScript!");
      </script>
```

```
</head>
<!-- ...  -->
```

Внешний JavaScript

Как правило, только простейшие скрипты помещаются в HTML документ. Если JavaScript-кода много – его выносят в отдельный файл, который, как правило, имеет расширение **.js**.

Чтобы включить в HTML-документ JavaScript-код из внешнего файла, нужно использовать атрибут **src** (source) тега script. Значением атрибута должен быть адрес файла, в котором содержится JS-код.

Например:

```
<script src = "/script/script.js"></script>
```

example_3. Внешний JavaScript

```
<!-- ...  -->
<head>
     <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
     <title>Введение в JavaScript</title>
     <!-- файл message.js должен присутствовать в директории -->
     <script src = "message.js"></script>
</head>
<!-- ...  -->
```

Для подключения нескольких скриптов используйте несколько тегов script:

```
<script src = "/script/banner.js"></script>
<script src = "/script/page.js"></script>
```



Одновременно подключить внешний файл и написать что-то внутри тега script нельзя. Придется создавать два разных тега: первый, с атрибутом **src**, второй - с **командами**, которые будут выполнены **после** выполнения внешнего файла.

```
<!-- так работать не будет -->
<script src = "/script/banner.js">
      alert("Добро пожаловать в JavaScript!");
</script>


<!-- а так будет -->
<!-- сначала выполнится внешний скрипт -->
<script src = "/script/banner.js"></script>
<!-- потом встроенный -->
<script>
      alert("Добро пожаловать в JavaScript!");
</script>
```

Внешние скрипты практичны, когда один и тот же JS-файл используется более чем в одной веб-странице проекта. Браузер скачает файл один раз, и в дальнейшем **будет брать его из своего кеша**, благодаря чему скрипт, содержащий, к примеру, библиотеку функций, может использоваться на разных страницах **без полной перезагрузки** с сервера.

> ***Важно***. У такой возможности браузера есть и оборотная сторона. Когда правите скрипты, не забывайте время от времени чистить кеш браузера (**Ctrl**+**Shift**+**Del**), либо выполняйте обновление страницы без учета кэшированного содержания (**Ctrl**+**F5**).

Кроме этого, благодаря внешним скриптам, упрощается сопровождение кода, поскольку вносить изменения или исправлять ошибки приходится только в одном месте.



JavaScript в атрибутах событий HTML-элементов

Не всегда нужно, чтобы выполнение сценария начиналось сразу при загрузке страницы. Чаще всего требуется, чтобы программа запускалась в ответ на **определенные действия пользователя**.

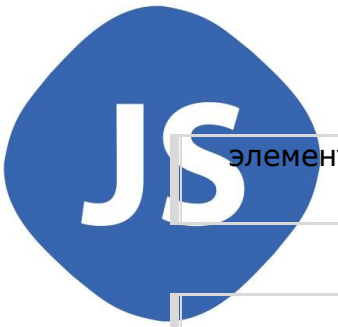
Каждый HTML-элемент имеет JS-события, которые срабатывают в определенный момент. Нужно добавить необходимое событие в HTML-элемент как атрибут, а в качестве значения этого атрибута указать исполняемый код (при дальнейшем изучении - исполняемую функцию). Код, исполняемый в ответ на срабатывание события, называется **обработчиком события**.

В следующем примере HTML-элемент **button** с атрибутом JavaScript события **onclick** обеспечивает реакцию скрипта на клик мыши. При нажатии кнопки генерируется событие **onclick** и выполняется связанный с событием JS-код.

example_4. JavaScript как обработчик события

```
<!-- ...  -->
      <header>
      <h1 align = "center">Введение в JavaScript</h1>
      </header>
      <button onclick = "alert('Добро пожаловать в JavaScript!')">Нажми
      меня</button>
      <footer>
            <h3 align = "center">Добро пожаловать на pechora_PROger</h3>
      </footer>
</body>
<!-- ...  -->
```

Важно. Независимо от того, как JS-код включается в HTML-документ, элементы script интерпретируются браузером в том порядке, в котором они **расположены в HTML-документе**. Сначала интерпретируется код первого



элемента script, затем браузер приступает ко второму элементу script и т. д.

Важно. События - тема отдельного урока. Здесь это понятие приведено для общего представления возможностей подключения сценариев.

Расположение тегов script

Как уже было сказано, браузер читает HTML-документ сверху вниз и, начинает отображать страницу, показывая часть документа до тега script. Встретив тег script, переключается в JavaScript-режим и выполняет сценарий. Закончив выполнение, возвращается обратно в HTML-режим и отображает оставшуюся часть документа.

Если на странице используется много скриптов JavaScript, то могут возникнуть длительные задержки при загрузке, в течение которых пользователь видит пустое окно браузера. Одним из способов решения проблемы считается указание JS-сценариев после контента страницы, перед закрывающим тегом body:

```
    <!-- ...  -->
    <script src = "/script/banner.js"></script>
    <script src = "/script/page.js"></script>
</body>
```

Такое расположение сценариев позволяет браузеру загружать страницу быстрее, так как сначала загрузится **контент** страницы, а потом будет загружаться **код** сценария.

Для пользователей это предпочтительнее, потому что страница полностью визуализируется в браузере до обработки JavaScript-кода.



Синхронные, асинхронные и отложенные сценарии

Расположение JS-сценариев перед закрывающим тегом body в некотором мысле ускоряет загрузку и отображение HTML-документа. Но есть более современные способы управлять порядком выполнения сценариев.

Синхронное выполнение сценария

В современных сайтах скрипты обычно тяжелее, чем код HTML: они весят больше, дольше обрабатываются.

По умолчанию файлы JS-кода прерывают синтаксический анализ (парсинг) HTML-документа до тех пор, пока скрипт не будет **загружен** (если он внешний) и **выполнен**, тем самым увеличивая промежуток времени до первой отрисовки страницы.

> ***Важно***. JavaScript является **блокирующим** ресурсом для парсера. Это означает, что JavaScript блокирует разбор самого HTML-документа. Когда парсер доходит до тега script (не важно внутренний он или внешний), он **останавливается**, забирает файл (если он внешний) и запускает его.

Это ведёт к двум важным проблемам:

- Скрипты не видят HTML-элементы ниже себя, поэтому к ним нельзя добавить обработчики и т.д.
- Если вверху страницы объёмный скрипт, он **блокирует** страницу. Пользователи не видят содержимое страницы, пока скрипт не загрузится и не запустится.

Такое поведение браузера называется **синхронным** и может вызвать некоторые проблемы со скоростью отображения сайта.

Рассмотрим пример кода **синхронного** выполнения внешнего скрипта.

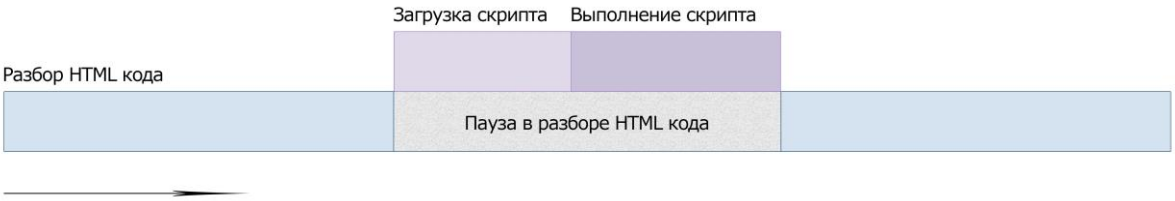


example_5. Синхронное выполнение скрипта

```
<html>
<head>
      <meta http-equiv="Content-Type" content="text/html; charset=utf-
      8">
      <title>Введение в JavaScript</title>
      <!-- файл sleep.js должен присутствовать в директории -->
      <script src = "sleep.js"></script>
</head>
<body style= "background-color: #0000ff">
<!-- ...  -->
```

В этом примере, пока браузер не загрузит и не выполнит sleep.js, он не покажет часть страницы под ним. Загрузка нескольких JavaScript-файлов на странице может привести к увеличению времени её отображения пользователю.

Схематически **синхронное** выполнение внешнего сценария можно представить следующим образом:



А что, если первичное отображение HTML-документа не зависит от загружаемых JS-файлов?

К сведению. Исполняемый скрипт может вносить изменения в первичную структуру HTML-документа, а может не вносить, и понимать это важно.

Кардинально решить проблему загрузки скриптов помогут атрибуты **async** и **defer** элемента script.

Атрибут async. Асинхронное выполнение

Async используется для того, чтобы указать браузеру, что скрипт может быть выполнен **асинхронно**. Атрибут доступен только для файлов, подключающихся **внешне**.

При обнаружении атрибута **async** браузер не останавливает обработку HTML-документа для загрузки скрипта, получение скрипта происходит **параллельно с разбором документа**. HTML-парсер будет приостановлен для выполнения скрипта, как только файл скрипта будет загружен.

Схематически **асинхронное** выполнение внешнего сценария можно представить диаграммой:



Рассмотрим пример кода **асинхронного** выполнения внешнего скрипта.

example_6. Асинхронное выполнение скрипта

```
<html>
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
    <title>Введение в JavaScript</title>
    <!-- файл sleep.js должен присутствовать в директории -->
    <script async src="http://js/sleep.js"></script>
</head>
```

```
<!-- ...  -->
```

Важно. В случае с атрибутами **async** и **defer** используйте полный URL-адрес в указании пути к скрипту.

```
<!-- для тестирования файлов курса я использую домен js -->
<!-- у вас это может быть другой адрес -->
<script async src="http://js/script1.js"></script>
```

Важно. Для сценариев с атрибутом **async** не гарантируется выполнение скриптов в порядке их добавления.

Исходя из сказанного, в примере:

```
<script async src="http://js/script1.js"></script>
<script async src="http://js/script2.js"></script>
```

второй скрипт может быть выполнен **перед первым**, поэтому важно, чтобы между этими сценариями **не было зависимостей**.

Атрибут defer. Отложенное выполнение

Атрибут **defer** откладывает выполнение скрипта до тех пор, пока вся HTML-страница не будет загружена полностью.

Как и при асинхронной загрузке скриптов — JS-файл может быть загружен, в то время как HTML-документ ещё грузится. Однако, даже если скрипт будет полностью загружен ещё до того, как браузер закончит обработку страницы, он **не будет выполнен** до тех пор, пока HTML-документ не обработается до конца.

Схематически **отложенное** выполнение внешнего сценария можно представить следующей диаграммой:



Рассмотрим пример кода **отложенного** выполнения внешнего скрипта.

example_7. Отложенное выполнение скрипта

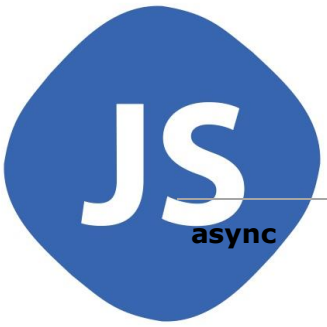
```
<html>
<head>
      <meta http-equiv="Content-Type" content="text/html; charset=utf-
      8">
      <title>Введение в JavaScript</title>
      <!-- файл sleep.js должен присутствовать в директории -->
      <script defer src="http://js/sleep.js"></script>
</head>
<body style="background-color: #0000ff">
<!-- ...  -->
```

Итого

У **async** и **defer** есть кое-что общее: они не блокируют отрисовку страницы. Так что пользователь может просмотреть содержимое страницы и ознакомиться с ней сразу же.

Примечание. В случае с атрибутом **async** точнее сказать так - он сводит к минимуму время блокировки HTML-парсера при отрисовке страницы.

Но есть и значимые различия:



	Порядок	Загрузка документа
async	*Порядок загрузки* (кто загрузится первым, тот и сработает)	Может загрузиться и выполниться до того, как документ загрузится. Такое случается, если скрипты маленькие или хранятся в кеше, а документ достаточно большой
defer	*Порядок документа* (как расположены в документе)	Выполняется после того, как документ загружен и обработан

На практике **defer** используется для скриптов, которым требуется доступ ко всему HTML-документу и/или важен их относительный порядок выполнения.

А **async** хорош для независимых скриптов, например счётчиков и рекламы, относительный порядок выполнения которых не играет роли.

Синтаксис JavaScript

Синтаксис JavaScript – это набор правил, по которым создаются программы JavaScript.

Набор символов

При написании программ на JavaScript используется набор символов **Unicode**. В отличие от 7-разрядной кодировки ASCII, подходящей только для английского языка, и 8-разрядной кодировки ISO Latin-1, подходящей только для английского и основных западноевропейских языков, 16-разрядная кодировка Unicode поддерживает практически **все письменные языки**, имеющиеся на планете.

example_8. Набор символов в скриптах неограничен

```
<script>
      // латиница
      var str = "Hello, World!";
```

```
    alert (str);
    // кириллица
    var строка = "Привет, Мир!";
    alert (строка);
</script>
```

Пробельные символы

Интерпретатор JavaScript игнорирует все пробельные символы которые могут присутствовать между языковыми конструкциями и воспринимает текст программы как **сплошной поток кода**.

Кроме того, JavaScript также, по большей части, игнорирует символы перевода строки. Поэтому пробелы и символы перевода строки могут без ограничений использоваться в исходных текстах программ **для форматирования и придания им удобочитаемого внешнего вида**.

Пробельные символы улучшают читаемость исходного кода, но эти символы, как правило, не нужны для функциональности JS-сценария.

Пробельные символы:

- горизонтальная табуляция;
- вертикальная табуляция;
- пробел;
- неразрывный пробел.

В дополнение к пробельным символам символы конца строки также используются для улучшения читаемости исходного текста. Однако в некоторых случаях символы конца строки могут влиять на выполнение кода JavaScript, поскольку есть несколько моментов, когда их применение запрещено. Символы конца строки также влияют на процесс автоматической установки точки с запятой.

Следующие символы распознаются интерпретаторами JavaScript как символы конца строки:



Символы конца строки:

- перевод строки;
- возврат каретки;
- разделитель строк;
- сепаратор абзацев.

Точка с запятой

Программа (сценарий) на языке JavaScript представляет собой перечень **инструкций**, которые выполняются веб-браузером.

Если несколько инструкций располагаются на одной строке, то между ними следует поставить знак "точка с запятой" (;).

```
<script>
    alert("Сообщение"); break;
</script>
```

Если каждая инструкция размещается на отдельной строке, то разделитель можно не писать. В таких случаях JavaScript интерпретирует переход на новую строчку как разделитель команд для **автоматического ввода точек с запятой** завершающих инструкции.

```
<script>
    alert("Сообщение")
    break
</script>
```

Одна инструкция может располагаться на нескольких строчках:

```
<script>
    var user = "Name";
    alert("Рады вас видеть, " +
        user +
        " !");
```



```
</script>
```

В этом случае JavaScript ждёт завершение выражения и поэтому автоматически не вставляет виртуальную точку с запятой между строчками.

Тем не менее, рекомендуется всегда добавлять точки с запятой – это позволит избежать побочных эффектов.

> ***К сведению***. Хотя точки с запятой в конце инструкций необязательны, рекомендуется всегда добавлять их. Сейчас это правило, которому следуют все большие проекты.

Чувствительность к регистру

Для написания JavaScript-программ используется **набор символов Unicode**, который включает в себя наборы ASCII и Latin-1 и поддерживается практически всеми языками и платформами.

В JavaScript все элементы, включая имена переменных, функций и операторов, **чувствительны к регистру** и должны всегда содержать одинаковые наборы прописных и строчных букв.

Например, ключевое слово **while** должно набираться как **while**, а не **While** или **WHILE**.

Аналогично **num**, **NUM** и **Num** – это три разные переменные.

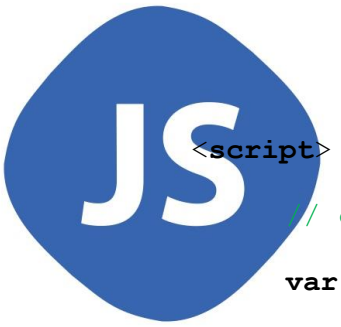
Комментарии

Комментарии позволяют выделить фрагмент программы, который не выполняется интерпретатором JavaScript, а служит лишь для пояснений содержания программы.

Комментарии в JS могут быть **однострочными** и **многострочными**.

Однострочные комментарии начинаются с двойного слэша (//). Текст считается комментарием до конца строки:





```
<script>
	// объявляем переменную
	var num = 5;
	alert( num );  // выводим переменную
</script>
```

Многострочный комментарий начинается с слэша и звездочки (/*), а заканчивается ими же в обратном порядке (*/). Так можно закомментировать одну и более строк:

```
<script>
	/* многострочный комментарий
	эта часть кода не выполнится
	var num = 5;
	alert( num );
	*/
	/* многострочный комментарий на одной строке */
	alert("Привет, Мир!");
</script>
```

К сведению. Не пренебрегайте комментариями в своих кодах. Они пригодятся вам при отладке и сопровождении программ. Есть мнение, что комментарии могут составлять до 30% от общего объема текста программ.

Идентификаторы

Идентификатор - это последовательность букв, цифр, символов подчёркивания (_) и знаков доллара ($). Цифра не может быть первым символом идентификатора, т. к. тогда интерпретатору JavaScript труднее отличать идентификаторы от чисел.

Идентификаторы выступают в качестве имён **переменных**, **функций**, **свойств объекта** и т. д.



Для совместимости и простоты редактирования для составления идентификаторов **обычно используются только символы ASCII и цифры**.

Однако в ECMAScript идентификаторы могут содержать буквы и цифры из полного набора символов Unicode. Это позволяет программистам давать переменным имена **на своих родных языках** и использовать в них математические символы.

> ***Примечание***. Хотя это и не запрещено стандартом, использовать русские символы в именах идентификаторов не принято .

Исторически, программисты использовали разные способы объединения нескольких слов для записи идентификаторов. Сегодня есть два устоявшихся негласных стиля:

- camelCase
- snake_case.

Например:

```
<script>
      var userLogin = "master"; // стиль camelCase
      var element_id = 5;  // стиль snake_case
</script>
```

В JavaScript наиболее популярным стилем именования идентификаторов, состоящих из нескольких слов, является camelCase – "верблюжья" нотация.

> ***Примечание***. В JavaScript объединение нескольких слов для записи идентификаторов с применением дефисов **запрещено**. Они зарезервированы для математических вычитаний.



Ключевые и зарезервированные слова

Стандарт ECMAScript определяет набор ключевых слов (keywords), которые не могут использоваться в качестве идентификаторов. Зарезервированные слова имеют определенное значение в языке JavaScript, так как они являются частью синтаксиса языка. Использование зарезервированных слов приведет к ошибке компиляции при загрузке скрипта.

Вот всего лишь некоторые зарезервированные ключевые слова: break, case, catch, class, while, continue, function...

ECMAScript

Чтобы понять, что такое ECMAScript и чем он отличается от JavaScript, нужно немного углубиться в терминологию.

Есть такая организация — Ecma International. Она создает стандарты для разных технологий. Самый простой пример стандарта — клавиатура **QWERTY**. Дизайн может быть любым, но расположение клавиш обычно QWERTY.

Ecma International разработала ECMA-262. Это **стандарт** скриптового языка.

В стандарте описана спецификация скриптового языка общего назначения. Эта спецификация называется **ECMAScript**.

> ***К сведению***. В спецификации ECMAScript содержатся рекомендации, сведения и правила, которые должны соблюдаться, чтобы скриптовый язык считался совместимым с ECMAScript

JavaScript — это скриптовый язык, который соответствует **спецификации ECMAScript**.

Таким образом:

- **Спецификация** ECMAScript расскажет, как создать свой скриптовый язык.



- **Документация** JavaScript объяснит, как пользоваться скриптовым языком.

В самостоятельных работах к уроку:

- Создание внутренних скриптов.
- Создание и подключение внешних скриптов
- Проверка работоспособности скрипта.

